Christian Javier Sosa Ruiz - 22000102
Ana Isabel Pérez Páiz - 22001411
Pablo Samuel Siquinajay Castro - 22006477

Ingeniería de Software Avanzada

Delivery 2: Governance & Technical Debt Audit

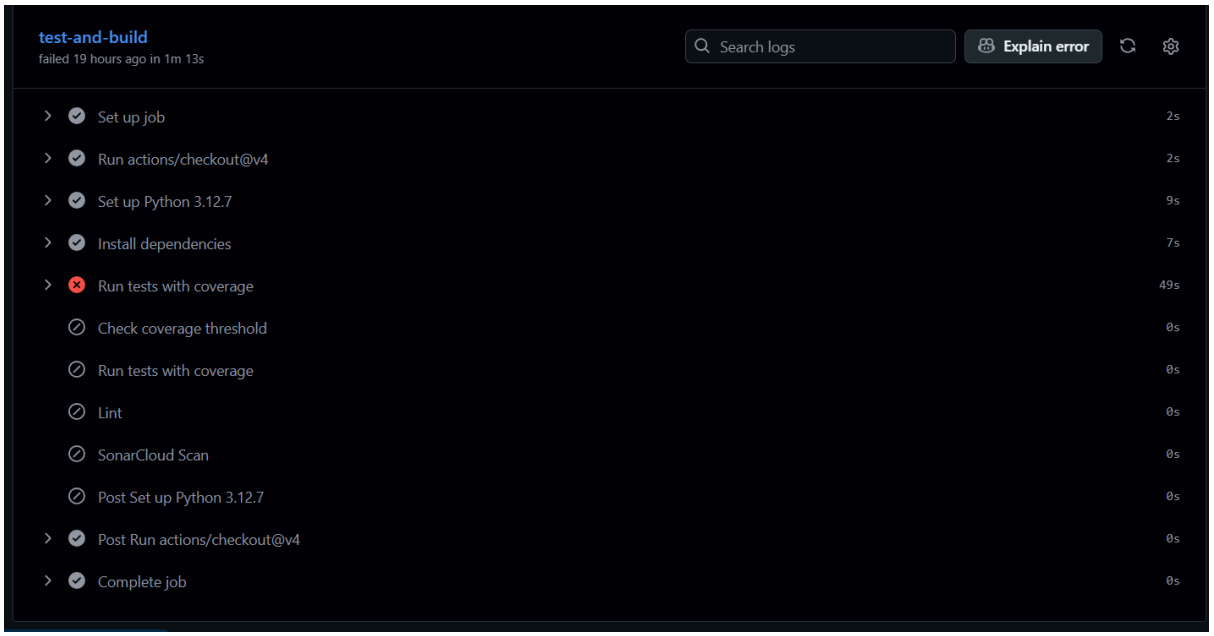
Guatemala, Ciudad de Guatemala 22 de Febrero de 2026

Governance Pipeline: **3**
○ CI Pipeline runs successfully on PRs. 4
○ Dashboard with four DORA metrics 4
Resumen de hallazgos principales 4
Resumen de Métricas DORA 4
Visualización 5
Limitaciones en la Visualización de Otras Métricas DORA 5
Tech Debt Audit **5**
Metodología 6
Hotspot #1 — `conduit/apps/articles/views.py` 6
Issues identificados 6
Análisis de ingeniería 8
Plan de refactor con Strangler Fig 8
Hotspot #2 — `conduit/apps/authentication/models.py` 11
Issues identificados 11
Análisis de ingeniería 12
Plan de refactor con Strangler Fig 12
Hotspot #3 — `conduit/apps/authentication/serializers.py` 13
Issues identificados 13
Análisis de ingeniería 14
Plan de refactor con Strangler Fig 15
Matriz de prioridad 16
Quality Gates 18
Definición de gates 18
Cómo interactúan los Quality Gates con la estrategia Strangler Fig 20
Mecanismo de enforcement 21
Configuración de SonarQube 21
Pasos de setup de SonarQube 22
Apéndice: Patrones de diseño referenciados 22
Apéndice: Principios SOLID referenciados 23

Governance Pipeline:

*Configure a CI workflow (GitHub Actions) that measures **Cyclomatic Complexity** and **Code Coverage**. It must fail/warn if metric degrades.*

CI Pipeline runs successfully on PRs. + SonarQube + Quality Gates

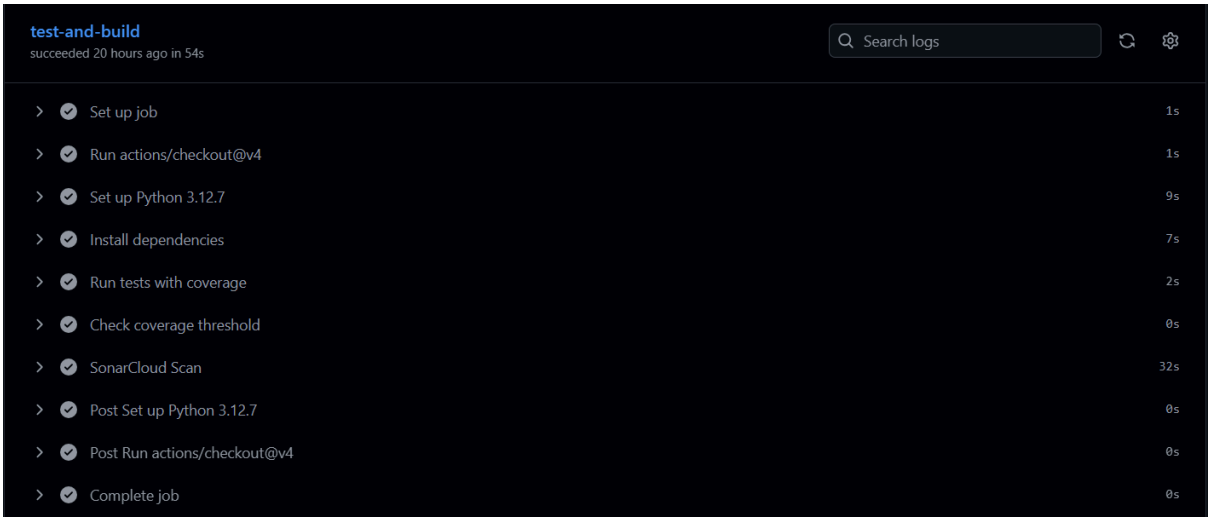


El CI pipeline se ejecuta completamente en cada pull request, ejecutando automáticamente análisis estáticos, test y coverage checks. Sin embargo, por causas más profundas con el código, el coverage no pudo ser evaluado ya que implicaba la creación de más archivos, lo que da indicios que no era problema de nuestros jobs del workflow, sino una **evidencia** de que el código le faltaba la agregación de **`tests.py`**

Ahora bien, si descartamos el análisis del coverage y nos vamos a ejecutar los siguientes jobs que evalúan el complexity analysis como éste,

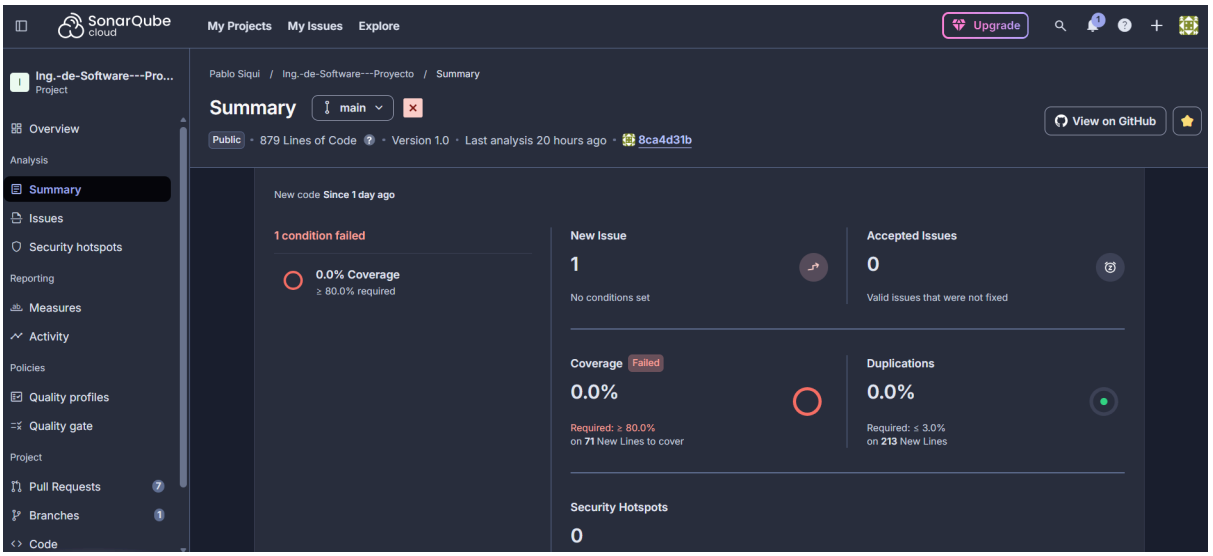
```
# Run the complexity analysis with eslint
    - name: Lint
      run: flake8 .
```

la instalación de independencias y SonarQube Scan



Vemos que sí se ejecutan todos los jobs.

SonarQube Cloud



SonarQube si reconoce los problemas de Coverage.
Los Quality Gates configurados se muestran más adelante

Dashboard with four DORA metrics

Antes de todo, se extrajo mediante AI-agent un archivo `dora-metrics-dashboard.json` que tiene toda la información del repositorio para el análisis de las métricas DORA.

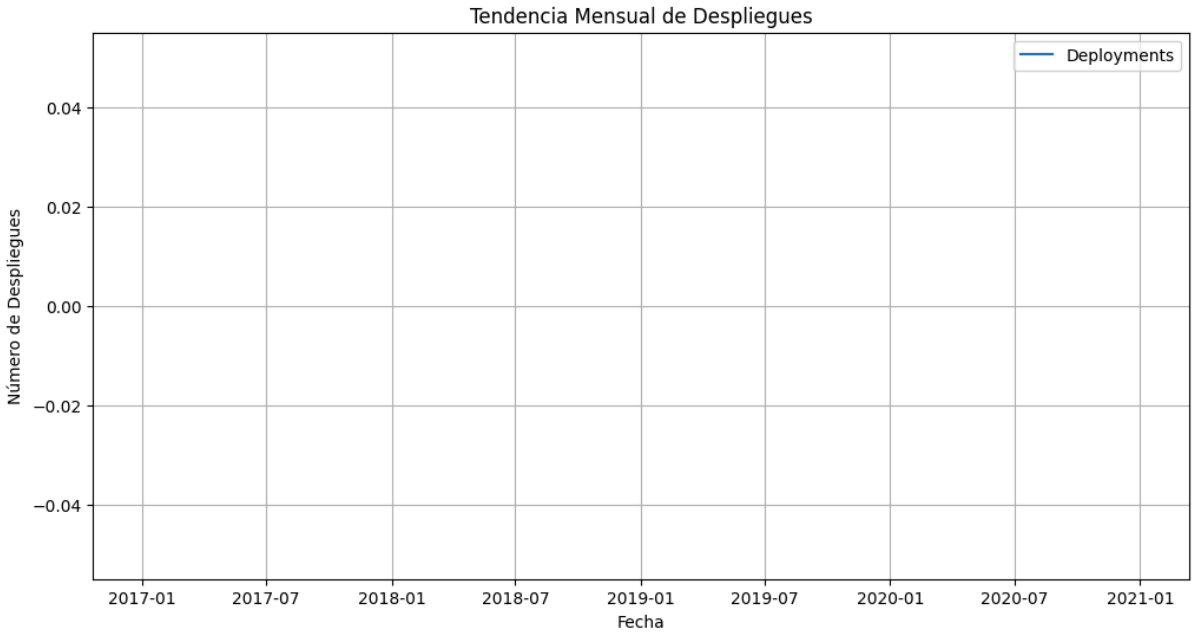
Resumen de hallazgos principales

- Solo 1 commit (inicial, del 7 de enero de 2019)
- 0 Pull Requests
- 0 Issues
- 0 Releases
- Sin CI/CD configurado

Resumen de Métricas DORA

- Deployment Frequency: No calculable (sin actividad)
- Lead Time for Changes: No calculable (sin PRs)
- Change Failure Rate: No calculable (sin issues/bugs)
- Mean Time to Recovery: No calculable (sin incidentes)

Visualización



Limitaciones en la Visualización de Otras Métricas DORA

Debido a la ausencia de datos en las series temporales para el 'Tiempo de entrega de cambios', la 'Tasa de fallos de cambios' y el 'Tiempo de restauración del servicio', no es posible generar gráficos de tendencias ni rendimiento para estas métricas. Los datos disponibles para estas métricas son nulos o vacíos en el archivo `dora-metrics-dashboard.json`.

¿Por qué los resultados son nulos?

Hay 5 razones estructurales que explican esto:

1. **El repositorio está archivado**

El repositorio fue marcado como archivado por gothinkster. Esto significa que es read-only: no se pueden mergear PRs, crear issues nuevos ni hacer commits. Es esencialmente un cadáver congelado en el tiempo (enero 2022).

2. **Nunca existió un pipeline CI/CD**

No hay ningún archivo en .github/workflows/, no hay .travis.yml, no hay Jenkinsfile, no hay nada. Esto significa que:

- Ningún PR tuvo checks automáticos
- No hay manera de medir Deployment Frequency ni Change Failure Rate
- Los PRs se aceptaban o rechazaban manualmente sin validación técnica automatizada

3. **Solo 1 PR fue mergeado en toda la historia**

De ~30 PRs totales, solo el #4 fue mergeado (2017, para actualizar Django 1.x). Todos los demás fueron cerrados sin merge. Esto hace que el Lead Time sea efectivamente infinito para la mayoría del trabajo propuesto.

4. **Documentación mínima y sin mantenimiento**

Existe un README.md básico, pero:

No hay CONTRIBUTING.md
No hay CHANGELOG
No hay documentación de arquitectura
No hay guía de despliegue
El README nunca fue actualizado para reflejar las versiones rotas de las dependencias
5. Dependencias críticas sin actualizar (Django 1.10.5)
El proyecto sigue usando Django==1.10.5 — una versión con End of Life desde diciembre 2017

Governance Impact - SonarQube

La integración de SonarQube permite la detección automatizada de:

- Errores
- Vulnerabilidades
- Objetivos de código
- Regresiones de cobertura; que ocurre cuando la cobertura de pruebas (tests unitarios, integración, etc.) disminuye respecto a una versión anterior**.**

Quality Gate aplica las políticas de gobernanza al bloquear el flujo de trabajo de CI cuando se superan los umbrales.

Tech Debt Audit

Tech Debt Audit: *Identify the top 3 "Hotspots" (files with high churn/complexity) and propose a refactoring plan using the Strangler Fig pattern.*

Metodología

los hotspots se identificaron mediante **análisis estático de complejidad**, evaluando:

- **Recuento de líneas y complejidad estructural** — número de clases, métodos y profundidad de anidamiento por archivo.
- **Code smells** — lógica duplicada, código muerto, inconsistencias de interfaz.
- **Violaciones SOLID** — incumplimientos del Principio de Responsabilidad Única (SRP), Principio Abierto/Cerrado (OCP) y Principio de Inversión de Dependencias (DIP).
- **Riesgos de seguridad** — faltas de validaciones de autorización, brechas de consistencia de datos.
- **Bugs de portabilidad y compatibilidad** — código dependiente de plataforma, incompatibilidades por versiones de librerías.

Se eligió el **patrón Strangler Fig** como estrategia de refactor para los tres hotspots. Este patrón, descrito originalmente por Martin Fowler, permite reemplazar código legacy de forma incremental construyendo implementaciones nuevas junto a las existentes, enrutando el tráfico hacia el código nuevo **un endpoint a la vez**, y retirando el código antiguo solo cuando la migración está completa. Este enfoque minimiza el riesgo porque:

1. El código antiguo se mantiene funcional durante toda la migración (cero downtime).

2. Cada paso de migración es testeable de forma independiente y reversible.

3. Evita el anti-patrón de “big bang rewrite”, que históricamente presenta una alta tasa de fallos.

Hotspot #1 — `conduit/apps/articles/views.py`

232 líneas - 6 clases - 12+ métodos

Riesgo: ALTO - Impacto: ALTO

Issues identificados

#	Issue	Categoría	Líneas afectadas
1	4 bloques duplicados `try/except Article.DoesNotExist` con mensajes de error casi idénticos	**Violación DRY**	L74-77, L90-93, L132-135, L169-172, L184-187
2	`ArticlesFavoriteAPIView.post` y `.delete` son casi idénticos (solo difieren en la llamada a `favorite`/`unfavorite` y el status code)	**Violación DRY**	L165-193
3	`ArticleViewSet.update` no hace verificación de ownership — cualquier usuario autenticado puede editar cualquier artículo	**Falla de seguridad (Broken Access Control — OWASP A01)**	L87-106
4	`CommentsDestroyAPIView.destroy` no hace verificación de ownership — cualquier usuario autenticado puede borrar cualquier comentario	**Falla de seguridad (Broken Access Control — OWASP A01)**	L149-157

5	El atributo de clase `queryset` en `ArticleViewSet` (L20) se sobreescribe en cada método vía `self.queryset.get()` — declaración muerta que confunde a quien lee	**Código muerto**	L20
6	6 clases de views fragmentadas sin una base común o mixin para patrones repetidos (lookup por slug, construcción de contexto del serializer)	**Falta de abstracción**	Archivo completo
7	`ArticleViewSet` usa `CreateModelMixin`, `ListModelMixin`, `RetrieveModelMixin` pero agrega `update` como método plano — contrato de interfaz inconsistente	**Inconsistencia de interfaz (preocupación LSP)**	L14-17 vs L87

Análisis de ingeniería

Por qué este es el hotspot de mayor prioridad:

Las fallas de seguridad (issues #3 y #4) corresponden a **Broken Access Control**, que es el riesgo #1 del OWASP Top 10 (2021). Cualquier usuario autenticado puede modificar o borrar contenido que no le pertenece. En un entorno de producción, esto sería una vulnerabilidad crítica.

Más allá de seguridad, los cinco bloques duplicados `try/except` violan el principio **DRY (Don't Repeat Yourself)**. Cada bloque sigue el mismo patrón:

```
try:
    article = Article.objects.get(slug=slug)
except Article.DoesNotExist:
    raise NotFound('An article with this slug does not exist.')
```

Esta duplicación incrementa la carga de mantenimiento: si cambia el formato del mensaje de error, o si se necesita agregar logging cuando falla el lookup, hay que actualizar 5 ubicaciones.

El `ArticlesFavoriteAPIView` también viola DRY: sus métodos `post` y `delete` (L165-193) comparten ~80% del código. Las únicas diferencias son:

- `profile.favorite(article)` **vs** `profile.unfavorite(article)`
- `HTTP_201_CREATED` **vs** `HTTP_200_OK`

Plan de refactor con Strangler Fig

Fase 1 — Crear módulo en paralelo (Bajo riesgo)

4. Crear `conduit/apps/articles/views_v2.py` junto al `views.py` existente.
5. Extraer un helper `get_article_by_slug(slug)` para eliminar los bloques duplicados `try/except`. Esto sigue el patrón de refactor **Extract Method**:

```
# views_v2.py

from rest_framework.exceptions import NotFound

from .models import Article


def get_article_by_slug(slug):

    """Lookup centralizado de artículos — elimina 5 bloques try/except duplicados."""

        try:

        return Article.objects.get(slug=slug)

        except Article.DoesNotExist:

        raise NotFound('An article with this slug does not exist.')
```

Fase 2 — Agregar autorización (Alto impacto, corrige la falla de seguridad)

6. Crear una clase de permisos `IsAuthorOrReadOnly` en `conduit/apps/articles/permissions.py`. Esto aplica el **Strategy Pattern**: el sistema de permisos de DRF permite cambiar estrategias de autorización de forma declarativa:

```
# permissions.py

from rest_framework.permissions import BasePermission, SAFE_METHODS


class IsAuthorOrReadOnly(BasePermission):

    """Permite operaciones de escritura solo si el usuario que hace la request es el autor."""

        def has_object_permission(self, request, view, obj):

        if request.method in SAFE_METHODS:

        return True

        return obj.author == request.user.profile
```

7. Aplicar `IsAuthorOrReadOnly` a `ArticleViewSet` (para `update`) y `CommentsDestroyAPIView` (para `destroy`).

Fase 3 — Consolidar favoritos (Impacto medio)

8. Refactorizar `ArticlesFavoriteAPIView` extrayendo un helper `_toggle_favorite`:

```
def _toggle_favorite(self, request, article_slug, action, success_status):

        profile = request.user.profile

        article = get_article_by_slug(article_slug)

        getattr(profile, action)(article)
```

```
serializer = self.serializer_class(article, context={'request': request})

return Response(serializer.data, status=success_status)
```

Fase 4 — Migración por rutas (Incremental)

9. Actualizar `conduit/apps/articles/urls.py` para enrutar endpoints a `views_v2.py` uno a la vez. Empezar por los endpoints críticos de seguridad (`update`, `destroy`).
10. Mantener `views.py` como fallback hasta que todas las rutas estén migradas.

Fase 5 — Limpieza

11. Eliminar `views.py` una vez que todas las rutas apunten a `views_v2.py`.
12. Renombrar `views_v2.py` a `views.py`.

Hotspot #2 — `conduit/apps/authentication/models.py`

139 líneas - 2 clases - 8 métodos

Riesgo: MEDIO-ALTO - Impacto: ALTO

Issues identificados

#	Issue	Categoría	Líneas afectadas
1	La lógica de generación de JWT (`_generate_jwt_token`) vive dentro del modelo `User`	**Violación SRP**	L127-139
2	`int(dt.strftime('%s'))` — el especificador de formato `%s` no forma parte del	**Bug latente (portabilidad)**	L136

	estándar C y **no funciona en Windows**		
3	`token.decode('utf-8')` — PyJWT >= 2.0 retorna `str` en lugar de `bytes`, causando `AttributeError`	**Bug de compatibilidad**	L139
4	La propiedad `token` re-genera el JWT en cada acceso — sin caching	**Smell de performance**	L100-109
5	En `conduit/apps/authentication/signals.py`, se usa el operador `is` para comparar enteros (`instance.pk is None` después de `.save()` puede no comportarse como se espera para enteros no cacheados)	**Preocupación de corrección**	signals.py

Análisis de ingeniería

Violación SRP (Single Responsibility Principle):

El modelo `User` tiene dos responsabilidades: (1) representar la identidad del usuario y gestionar el estado de autenticación, y (2) generar tokens JWT. Según SRP, una clase debería tener una sola razón para cambiar. Actualmente, el modelo `User` debe cambiar si cambia el esquema del usuario *o* si cambia la estrategia de generación del token (por ejemplo, cambiar de HS256 a RS256, agregar claims, cambiar expiración).

Bug de portabilidad:

La llamada `strftime('%s')` (L136) depende de una extensión específica de plataforma. En Windows, esto lanza `ValueError` porque `%s` no es una directiva reconocida. La alternativa portable es `datetime.timestamp()` (Python 3.3+) o `calendar.timegm()`.

Compatibilidad con PyJWT:

A partir de PyJWT 2.0 (lanzado en enero de 2021), `jwt.encode()` retorna `str` en lugar de `bytes`. Llamar `.decode('utf-8')` sobre un `str` lanza `AttributeError: 'str' object has no attribute 'decode'`. Este breaking change se manifestaría inmediatamente al actualizar PyJWT.

Plan de refactor con Strangler Fig

Fase 1 — Crear TokenService (Bajo riesgo)

13. Crear `conduit/apps/authentication/services.py` con una clase `TokenService`. Esto aplica el refactor **Extract Class** y sigue el **Service Layer Pattern**: la lógica de dominio (generación JWT) se mueve fuera del modelo de datos hacia un servicio dedicado:

Fase 2 — Bridge por delegación (cero ruptura)

14. Actualizar la propiedad `User.token` para delegar a `TokenService`:

```
@property
def token(self):
    from .services import TokenService
    return TokenService.generate_token(self)
```

Esto es el “strangler bridge”: los consumidores externos siguen accediendo a `user.token` sin cambios, pero la implementación ahora vive en `TokenService`.

Fase 3 — Migrar callers (Incremental)

15. Actualizar serializers (`RegistrationSerializer`, `LoginSerializer`, `UserSerializer`) para llamar `TokenService.generate_token(user)` directamente en lugar de `user.token`.

Fase 4 — Limpieza

16. Cuando todos los callers usen `TokenService`, eliminar la propiedad `token` y el método `_generate_jwt_token` del modelo `User`.

Hotspot #3 — `conduit/apps/authentication/serializers.py`

177 líneas - 3 clases - 4 métodos

Riesgo: MEDIO - Impacto: MEDIO-ALTO

Issues identificados

#	Issue	Categoría	Líneas afectadas
1	`LoginSerializer.validate` realiza autenticación + autorización + construcción de respuesta en un solo método	**Violación SRP**	L42-96
2	`UserSerializer.update` llama `instance.save()` (L166) y `instance.profile.save()` (L174) sin `@transaction.atomic` — si el guardado del profile falla, los datos del usuario quedan parcialmente comprometidos	**Riesgo de consistencia de datos**	L139-176
3	`from conduit.apps.profiles.serializers import ProfileSerializer` — import entre apps genera acoplamiento fuerte entre `authentication` y `profiles`	**Smell de acoplamiento (violación DIP)**	L5
4	El campo `profile` se declara `write_only=True` (L115) mientras `bio` e `image` son `read_only=True` (L119-120) — representación bidireccional de lectura/escritura del mismo dato a través de rutas de campos distintas	**Diseño de API confuso**	L115-120

Análisis de ingeniería

Violación SRP en `LoginSerializer.validate`:

Este método de 55 líneas (L42-96) maneja tres responsabilidades distintas:

17. **Validación de entrada** — verificar que email y password estén presentes.
18. **Autenticación** — llamar a `django.contrib.auth.authenticate()`.
19. **Construcción de respuesta** — construir el diccionario de retorno con email, username y token.

Según SRP, cada una debería ser una preocupación separada. El método `validate` debería limitarse a validar entrada; la autenticación debería delegarse a un servicio; y la construcción de la respuesta es una preocupación de representación (`to_representation`) del serializer.

Riesgo de consistencia de datos:

En `UserSerializer.update` (L139-176), primero se guarda el usuario (L166) y luego se guarda el profile (L174). Si el guardado del profile lanza una excepción (por ejemplo, una violación de constraint en base de datos), los cambios del usuario ya quedaron commitados. Esto viola la propiedad de **Atomicidad**: o se guardan ambos cambios, o no se guarda ninguno. La corrección es directa: envolver el método en `@transaction.atomic`.

Acoplamiento entre apps:

El import de `ProfileSerializer` desde `conduit.apps.profiles` (L5) crea una dependencia directa del app `authentication` hacia el app `profiles`. Esto viola el **Principio de Inversión de Dependencias (DIP)**: los módulos de alto nivel (authentication) no deberían depender de módulos de bajo nivel (profiles). En un proyecto Django bien organizado por capas, las apps deberían comunicarse por interfaces bien definidas o señales de Django, no por imports directos de serializers de otras apps.

Plan de refactor con Strangler Fig

Fase 1 — Crear AuthenticationService (Bajo riesgo)

20. Agregar un método `authenticate(email, password)` a `services.py` (creado en el Hotspot #2):

Fase 2 — Crear serializers v2 (Bajo riesgo)

21. Crear `conduit/apps/authentication/serializers_v2.py`.

22. Implementar `LoginSerializerV2.validate`: solo valida formato de entrada y delega a `AuthenticationService.authenticate()`:

Fase 3 — Migración por rutas (Incremental)

24. Actualizar `LoginAPIView` para usar `LoginSerializerV2` y `UserRetrieveUpdateAPIView` para usar `UserSerializerV2`.

Fase 4 — Limpieza

25. Eliminar las clases antiguas de serializers una vez que todas las views usen los serializers v2.

26. Renombrar `serializers_v2.py` a `serializers.py`.

Matriz de prioridad

Los hotspots se priorizan usando una matriz **Riesgo × Impacto**, donde:

- **Riesgo** mide la severidad de los problemas existentes (fallas de seguridad > bugs de compatibilidad > code smells).
- **Impacto** mide cuántas features/endpoints se ven afectados y el radio de explosión de una falla.

Prioridad	Hotspot	Riesgo	Impacto	Justificación

P0 — Crítico	`articles/views.py`	ALTO	ALTO	Contiene vulnerabilidades de seguridad activas (faltan ownership checks en update y delete). Mayor complejidad estructural (6 clases, 12+ métodos). Mayor duplicación (5 bloques `try/except` repetidos). Afecta la mayor cantidad de endpoints (8 de 14 endpoints totales).
P1 — Alto	`authentication/models.py`	MEDIO-ALTO	ALTO	Contiene bugs de compatibilidad que fallan en Windows (`strftime('%s')`) y en PyJWT >= 2.0 (`.decode('utf-8')`). La violación SRP en el modelo core `User` hace que cualquier cambio de tokens requiera modificar el modelo. Alto impacto porque la generación de JWT afecta todas las requests autenticadas.

P2 — Medio	`authentication/serializers.py`	MEDIO	MEDIO-ALTO	Riesgo de consistencia de datos (doble save no atómico) y violaciones SRP. Menor radio de explosión que P0/P1 porque las fallas se concentran en login y user-update. El acoplamiento entre apps es un problema de mantenibilidad, pero no un riesgo inmediato de runtime.

Quality Gates

Los quality gates definen los **criterios mínimos de calidad** que el código nuevo debe cumplir antes de poder fusionarse (merge) a la rama principal. Estos gates se hacen cumplir mediante SonarQube como parte del **pipeline de CI/CD**.

Definición de gates

Gate	Métrica	Umbral	Justificación

Confiabilidad (Reliability)	Bugs nuevos en el código modificado	**0**	No se permiten bugs nuevos en ningún PR. Es el umbral más estricto posible: tolerancia cero a regresiones.
Seguridad (Security)	Vulnerabilidades nuevas en el código modificado	**0**	No se permiten nuevos issues de seguridad. Esto atiende directamente los hallazgos de Broken Access Control en el Hotspot #1.
Mantenibilidad (Maintainability)	Ratio de deuda técnica en el código nuevo	**≤ 5%**	Permite enfocarse en “estrangular” la deuda del legacy sin quedar bloqueados por ella, asegurando que todo el código nuevo sea limpio. El 5% significa que por cada 100 líneas nuevas, como máximo 5 líneas pueden acumular code smells sin resolver.
Duplicación	Líneas duplicadas en el código nuevo	≤ **3%**	Hace cumplir DRY en contribuciones nuevas. Este umbral es más estricto que el default de SonarQube (que a menudo ni está configurado) porque la duplicación es el problema más prevalente encontrado en esta auditoría.

Cobertura (Coverage)	Cobertura de líneas en el código nuevo	**≥ 80%**	Dado que el proyecto actualmente tiene cobertura de pruebas cero, este gate obliga a que todo código nuevo venga testeado. 80% es un estándar de industria que balancea rigurosidad con practicidad (algunas líneas como handlers de excepciones son difíciles de cubrir).
Complejidad (Complexity)	Complejidad cognitiva por método	**≤ 15**	Evita la creación de nuevos hotspots. La complejidad cognitiva de SonarQube mide qué tan difícil es entender un método (considera anidamiento, rupturas del flujo lineal y recursión). Un umbral de 15 es la recomendación por defecto de SonarQube.

Cómo interactúan los Quality Gates con la estrategia Strangler Fig

Los quality gates se configuran para evaluar **solo el código nuevo** (no toda la base de código). Esto es crítico porque:

27. El código legacy (que estamos analizando) fallaría múltiples gates (por ejemplo, `LoginSerializer.validate` tiene complejidad cognitiva > 15 y hay 0% de cobertura de pruebas).

28. Al acotar los gates al código nuevo, el equipo puede fusionar refactors tipo Strangler Fig de forma incremental sin que el legacy bloquee el pipeline.

29. A medida que el código legacy se reemplaza, las métricas globales de la base de código mejoran de manera orgánica.

Mecanismo de enforcement

Los quality gates se hacen cumplir en dos niveles:

30. **Servidor SonarQube** — El quality gate se configura en la UI de SonarQube bajo *Quality Gates > Create*. El gate se aplica al proyecto `conduit-realworld-django`.

31. **Pipeline de CI** — `sonar-scanner` se ejecuta como un paso dentro del pipeline (por ejemplo, GitHub Actions). Si el quality gate falla, el pipeline falla y el PR no puede ser mergeado.

Configuración de SonarQube

El proyecto se configura mediante `sonar-project.properties` en la raíz del repositorio. Decisiones clave de configuración:

Propiedad	Valor	Justificación

`sonar.projectKey`	`conduit-realworld-django`	Identificador único del proyecto en SonarQube.
`sonar.sources`	`conduit`	Todo el código de la aplicación vive bajo el directorio `conduit/`.
`sonar.language`	`py`	El proyecto es una aplicación Python (Django).
`sonar.exclusions`	`**/migrations/**,**/tests/**,manage.py`	Las migrations de Django son auto-generadas y no deberían analizarse. Los tests tienen estándares de calidad distintos. `manage.py` es boilerplate.

Pasos de setup de SonarQube

32. **Instalar SonarQube** (Community Edition).
33. **Crear el proyecto** en la UI de SonarQube con key `conduit-realworld-django`.
34. **Configurar el quality gate** usando los umbrales definidos arriba.
35. **Generar un token de autenticación** para el pipeline de CI.
36. **Agregar `sonar-project.properties`** a la raíz del repositorio (ya hecho — ver archivo en la raíz del repo).
37. **Integrar con CI** — agregar un paso `sonar-scanner` al workflow de GitHub Actions (entregable separado).

Apéndice: Patrones de diseño referenciados

Patrón	Dónde se aplica	Descripción
Strangler Fig	Todos los hotspots	Reemplazo incremental del legacy construyendo implementaciones nuevas junto a las antiguas, migrando tráfico y retirando el código viejo. Evita los big-bang rewrites.
Strategy Pattern	Permiso `IsAuthorOrReadOnly`	El sistema de permisos de DRF usa Strategy: las clases de permisos son estrategias intercambiables para decisiones de autorización.
Service Layer Pattern	`TokenService`, `AuthenticationService`	Extrae lógica de dominio desde modelos/serializers hacia servicios dedicados, siguiendo SRP.
Extract Method	`get_article_by_slug()`	Consolida código duplicado en un método único y reutilizable.
Extract Class	`TokenService` desde `User`	Mueve un conjunto cohesivo de responsabilidades desde una clase hacia una clase nueva y dedicada.

Apéndice: Principios SOLID referenciados

Principio	Violaciones encontradas	Hotspot
SRP (Single Responsibility)	Generación de JWT en el modelo User; auth +	#2, #3

	validación + respuesta en `LoginSerializer.validate`	
OCP (Open/Closed)	Agregar nuevas acciones de artículos requiere modificar clases de views existentes	#1
LSP (Liskov Substitution)	`ArticleViewSet` declara mixins pero agrega `update` fuera del contrato de mixins	#1
DIP (Dependency Inversion)	Import directo entre apps de `ProfileSerializer` en serializers de authentication	#3